

Full Stack Developer Assignment

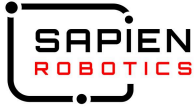
Instructions: Send us a GitHub repo with your code and a README that explains how to run it. Use whatever language and framework you like. Using AI assistants is fine, just be ready to talk through any part of what you submit.

Question 1: Multiplayer Drawing & Guessing Game (Pictionary-style)

Build a real-time multiplayer game in the browser, similar to Skribbl.io. One player gets a secret word and draws it. Everyone else guesses in chat, and faster correct guesses earn more points. The turn then passes to the next player.

Requirements:

- Rooms you can create and join through a shareable link, with a host who sets the number of rounds, draw time and word list.
- Drawing tools: pen, colours, brush size, eraser, fill, undo and clear canvas. Strokes stream to every player as they are drawn, with no visible lag.
- Turn flow: the drawer picks from 3 words, then a countdown timer starts. Letter hints reveal over time. The round ends when time runs out or when everyone has guessed.
- The server is the authority. The server alone checks guesses, keeps time and awards points. A player who edits the client must not be able to see the word or give themselves points.
- Players who guess right can't reveal the word in chat. Near-misses show "close!" to the person who guessed.
- A player who joins mid-round sees the canvas as it stands. A player who refreshes or drops for up to 30 seconds rejoins with their score kept.
- When the drawer leaves mid-turn, the game recovers cleanly instead of getting stuck.
- Basic abuse protection: rate limiting on chat and strokes, and limits on how large a message or stroke can be.
- A stroke-by-stroke replay of every drawing at the end of the game.



Deliverables:

- The game running from one documented command.
- A two-minute screen recording with at least 3 players in separate browsers.
- A short note covering:
 - how you sync the canvas and send stroke data
 - how you handle latency and ordering
 - what happens when the server restarts
 - roughly how many rooms one server instance can handle, and how you measured it

Question 2: Music Streaming with Shared Listening Rooms (Architecture)

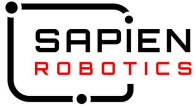
Design the backend and system architecture for a music streaming service in the style of Spotify, with a "Listen Together" room feature. A user creates a room and others join with a link. Everyone hears the same track at the same moment. Anyone can add to a shared queue and vote to skip, and the host can pause and seek for the whole room. You don't need to write code for this question. Include a short proof of concept only if it helps explain your design.

Assume:

- 20M monthly users, with 2M listening at the same time at peak.
- A catalogue of 50M tracks.
- 200k shared rooms open at peak, from 2 to 5,000 listeners per room.

Your design should cover:

- *Audio delivery*: storage, encoding, CDN, adaptive bitrate, fast playback start.
- *Catalogue and search*: metadata storage, search as you type, publishing new releases.
- *Room sync*: keeping every listener within about 200ms, late joiners, slow connections, failover when the node holding a room dies.
- *Data and storage*: what goes in which database, and why. Listening history, playlists, room state, events for royalties and analytics.
- *Reliability*: behaviour during a regional outage, deploys that don't drop live rooms, and what the system does when parts of it are struggling.
- *Observability*: the metrics that tell you whether sync is actually working for users.



Deliverables:

- An architecture diagram, plus a sequence diagram for "host presses pause in a room of 5,000 listeners".
- A design doc of 3–6 pages:
 - your key decisions, with alternatives you rejected
 - at least 5 failure scenarios and how the system handles each
 - what you'd build first with a team of three, and what you'd postpone

Submission Guidelines:

1. Email the deliverables to aditya.patil@sapienrobotics.ai